Guía de patrones de diseño en Java

Material de estudio para Tomás Garayzar. Preparado a partir de la teoría, las consignas y las soluciones de GaryPoui/Ejercicios-Java, revisadas el 26 de octubre de 2026.

Qué tenés que dominar

La prioridad de la evaluación, según lo que indicás, es Observer, Adapter, Decorator y Composite. También se incluyen Builder, Strategy y State como contenidos posibles. No alcanza con recordar el dibujo: tenés que reconocer el problema, asignar responsabilidades, dibujar las relaciones y explicar qué ocurre al ejecutar el código.

Cómo usar esta guía

• Primera pasada: estudiá los fundamentos y los cuatro patrones principales. Después explicá cada diagrama en voz alta sin leer.

• Segunda pasada: seguí los recorridos de llamadas y resolvé los cálculos de stock, precios e impuestos a mano.

• Tercera pasada: repasá Builder, Strategy y State y resolvé los cinco ejercicios del PDF de patrones IV.

• Antes de rendir: hacé el simulacro sin mirar las respuestas y corregí tanto la elección del patrón como las relaciones del modelo.

Qué vas a encontrar

Recorrido por páginas: 2 y 3 fundamentos; 4 a 7 Observer y Adapter; 8 a 12 Composite y Decorator; 13 a 16 Builder, Strategy y State; 17 y 18 exportador y consorcios; 19 a 22 patrones IV; 23 comparaciones; 24 y 25 simulacro; 26 repaso y fuentes.

Fundamentos y UML; siete patrones con diagramas y Java; resolución de stock y precios, Frávega, exportador y consorcios; los cinco casos de patrones IV; comparaciones, errores frecuentes, simulacro y respuestas.

Cómo interpretar las soluciones

Cuando se describe el código del repositorio, se indica su carpeta o archivo. Las propuestas didácticas agregan soluciones o mejoras para estudiar; no se presentan como soluciones oficiales del docente. Las consignas están resumidas, conservando sus reglas relevantes.

Base revisada: commit 4aa08b79e08552d71d727bbc8e99c7cd12f66f54. Los PDF de teoría y consignas repetidos en PruebaTincho tienen el mismo contenido que los de la raíz. Los fragmentos Java son ejemplos didácticos; no se verificó su compilación en este entorno.

Fundamentos para resolver la prueba

Un patrón es una decisión de diseño

Un patrón describe una solución reutilizable para un tipo de problema. No es una biblioteca ni una clase que se copia siempre igual. Dos soluciones pueden usar nombres distintos y representar el mismo patrón si distribuyen las responsabilidades de la misma forma.

Interfaz: contrato de operaciones. Clase abstracta: contrato que además puede compartir atributos y comportamiento. Polimorfismo: una referencia del contrato puede apuntar a diferentes implementaciones. Delegación: un objeto pide a otro que haga parte del trabajo.

```
Item item = new Producto("Casco", 100, 5);
item = new Packaging(item, 50);
System.out.println(item.calcularPrecio()); // 150
```

La variable sigue siendo Item, aunque el objeto exterior ahora sea Packaging. El cliente pregunta el precio al contrato y cada objeto resuelve su parte.

Principios que sirven para justificar

• Alta cohesión: cada clase concentra una responsabilidad clara. Por ejemplo, vender y registrar en una API contable se separan.

• Bajo acoplamiento: el cliente depende de contratos estables. Esto reduce su dependencia de implementaciones concretas; no elimina toda dependencia del sistema.

• Abierto a extensión: agregar una reacción, un algoritmo o una capa debería requerir pocas modificaciones al código existente.

• Composición: un objeto conserva referencias a colaboradores. Evita crear una subclase por cada combinación posible de características.

Cómo leer los gráficos

Un triángulo vacío apunta al tipo general. Línea continua con triángulo: herencia, equivalente a extends. Línea discontinua con triángulo: realización de una interfaz, equivalente a implements. Flecha simple: un objeto conoce, contiene o usa al otro; el texto de la relación aclara cuál.

Una colección de componentes se representa con un atributo como List<Item>. No dibujes herencia para expresar que un combo contiene productos: contiene y es un tipo de son relaciones diferentes.

Método para cualquier enunciado

1. Subrayá lo que cambia. 2. Identificá qué operación necesita el cliente. 3. Definí un contrato. 4. Separá las implementaciones. 5. Dibujá las referencias. 6. Recorré un caso concreto. 7. Declará los supuestos que la consigna no define.

Mapa para reconocer los siete patrones

Patrón y familia	Problema que resuelve	Señal en una consigna
Observer · comportamiento	Un evento activa varias reacciones desacopladas.	Cada vez que ocurre X, avisar a interesados.
Adapter · estructural	Una interfaz disponible no coincide con la requerida.	Integrar biblioteca o sistema externo.
Decorator · estructural	Agregar capas acumulables manteniendo el contrato.	Agregar o quitar funciones en ejecución.
Composite · estructural	Tratar hojas y agrupaciones con la misma interfaz.	Grupos de grupos; estructura recursiva.
Builder · creacional	Construir por pasos y entregar un objeto válido.	Muchos datos y reglas de consistencia.
Strategy · comportamiento	Intercambiar algoritmos para una misma tarea.	Elegir criterio o formato en ejecución.
State · comportamiento	Cambiar comportamiento según el estado actual.	Acciones válidas y transiciones de estados.

La intención decide

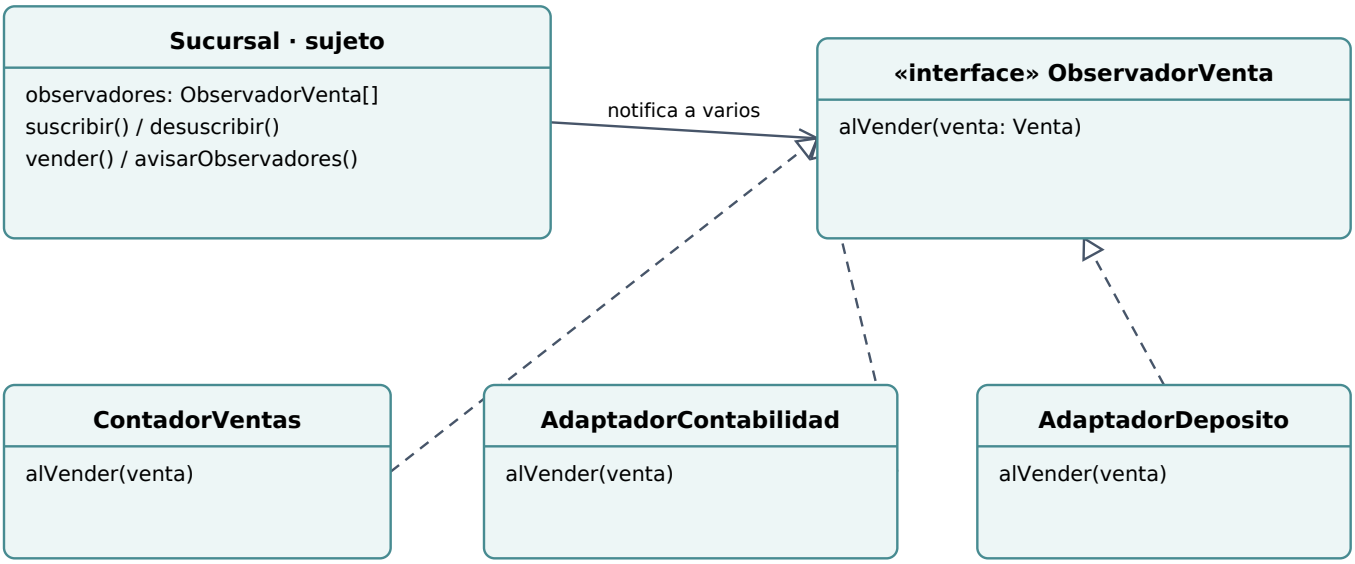
Tener una interfaz y varias clases no alcanza para nombrar un patrón. Strategy y State pueden verse parecidos en UML, pero Strategy representa alternativas de un algoritmo y State representa situaciones del ciclo de vida. Tener una lista tampoco demuestra Composite: la lista debe permitir agrupar componentes y consultarlos con un contrato común de forma recursiva.

Podés combinar patrones

En Frávega, Observer organiza el aviso de una venta y Adapter traduce el aviso hacia sistemas externos. En stock, Composite arma los combos y Decorator agrega descuentos y packaging. En el exportador, Strategy elige Excel o PDF y Adapter aísla la biblioteca PDF. Cada patrón resuelve una variación distinta.

Observer explica quién reacciona a un evento

Intención. Separar el objeto donde ocurre un evento de los interesados que reaccionan. El sujeto conserva observadores del mismo contrato y los notifica cuando corresponde. Es un patrón de comportamiento.



Roles en Frávega

Sujeto: Sucursal. Observador: ObservadorVenta. Reacciones: registrar el importe, preparar el producto y contar ventas. Evento: Venta, que lleva los datos necesarios. La sucursal sabe que existe alVender, pero no necesita programar los detalles de cada reacción.

Secuencia que tenés que poder contar

Se suscriben los tres interesados. Al vender, la sucursal crea y guarda una Venta. Después recorre sus observadores y llama alVender(venta). Contabilidad usa el importe; depósito usa el código; el contador registra el evento para su conteo diario. Al desuscribir un interesado, las ventas futuras dejan de avisarle.

Qué aporta y qué cuesta

Permite agregar un observador sin meter otro bloque de lógica en vender. También facilita localizar cada reacción. Como contrapartida, el flujo queda repartido entre varios objetos y hay que decidir qué hacer ante errores y suscripciones repetidas.

El apunte describe notificaciones sincrónicas y secuenciales. Observer por sí mismo no crea hilos, paralelismo ni garantías de entrega. Un aviso por email no define Observer: lo define la relación entre el evento, el sujeto y sus interesados.

Observer en Java y recorrido de una venta

Propuesta didáctica simplificada. Importaciones necesarias: java.util.ArrayList y java.util.List. Venta es el evento del dominio, como en el repositorio.

```
interface ObservadorVenta {
    void alVender(Venta venta);
}
class Sucursal {
    private final List<ObservadorVenta> observadores =
        new ArrayList<>();
    void suscribir(ObservadorVenta o) {
        if (!observadores.contains(o)) observadores.add(o);
    }
    void desuscribir(ObservadorVenta o) {
        observadores.remove(o);
    }
    void notificar(Venta v) {
        for (ObservadorVenta o : new ArrayList<>(observadores)) {
            o.alVender(v);
        }
    }
}
class Contador implements ObservadorVenta {
    private int cantidad;
    public void alVender(Venta v) { cantidad++; }
    int cantidad() { return cantidad; }
}
```

La lista se tipa con la interfaz. El for llama el mismo método y Java selecciona la implementación real. La copia de la lista evita que una baja o alta durante la notificación modifique ese recorrido; no convierte el código en seguro para varios hilos.

Ejercicio corto resuelto

Se suscriben A, B y C. Se vende un producto. Se elimina B. Se venden otros dos. ¿Cuántas veces recibe cada uno el aviso? A: 3; B: 1; C: 3. El evento de baja afecta las notificaciones posteriores, no borra las recibidas.

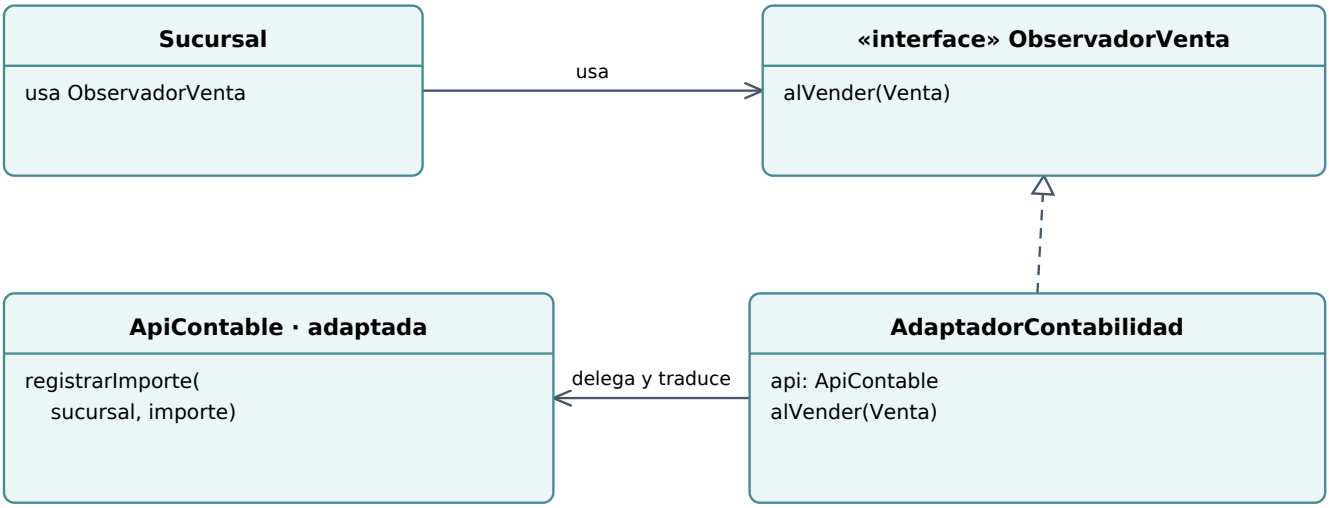
Qué hace el código real

Sucursal.java usa arreglos redimensionables en lugar de List. Guarda la venta antes de avisar, toma una copia de destinatarios y captura RuntimeException por observador para continuar con los demás. Esa decisión evita que un fallo frene el recorrido, pero no reintenta el aviso fallido ni revierte la venta.

Venta almacena el importe al crearse. Si después cambia el IVA, una venta anterior conserva su importe. Esta es una regla de dominio útil para estudiar: el evento debe representar lo ocurrido, sin recalcular su valor histórico con tasas nuevas.

Adapter traduce una interfaz incompatible

Intención. Encapsular una clase o biblioteca detrás del contrato que necesita el cliente. Es estructural. El adaptador recibe una llamada propia del dominio y la convierte en las llamadas, parámetros o formatos de la dependencia externa.




```
class AdaptadorContabilidad implements ObservadorVenta {
    private final ApiContable api;
    AdaptadorContabilidad(ApiContable api) {
        this.api = api;
    }
    public void alVender(Venta venta) {
        api.registrarImporte(
            venta.getSucursal(), venta.getImporte());
    }
}
```

Contrato requerido: ObservadorVenta. Adaptador: AdaptadorContabilidad. Adaptado: ApiContable. El cliente no tiene que aprender registrarImporte; el adaptador extrae de Venta los datos que necesita la API.

Ejercicio corto resuelto

Una API nueva exige registrar(String local, long centavos). ¿Qué cambiás? Creás otro adaptador del mismo contrato o modificás el actual si realmente reemplaza a la API anterior. La conversión a centavos queda allí, con una política explícita de redondeo. Sucursal y ContadorVentas conservan sus responsabilidades.

Qué no promete el patrón

No elimina todos los errores de red ni asegura que dos servicios tengan la misma semántica. La teoría habla de desacoplar cliente y clase adaptada; la dependencia sigue existiendo dentro del adaptador. En el repositorio, ApiContable y ApiDeposito imprimen mensajes: representan las integraciones, no implementan llamadas HTTP reales.

Frávega resuelto con Observer y Adapter

Consigna resumida. Una venta debe avisar a contabilidad, depósito y contador diario. Además, cada categoría aplica IVA y otros impuestos extensibles. Fuente: 6-Frávega (1).pdf y Fravega_Java (2)/fravega.

Modelo y responsabilidades

• Sucursal crea las ventas y avisa mediante ObservadorVenta. No calcula cómo registrar un asiento contable ni preparar un pedido.

• AdaptadorContabilidad y AdaptadorDeposito son observadores y adaptadores a la vez: cumplen el contrato del aviso y traducen hacia APIs distintas.

• ContadorVentas es observador sin adaptar una API. Cuenta eventos por fecha.

• Producto calcula precioBase más la suma de cada Impuesto aplicado sobre precioBase. TipoProducto reúne los impuestos y exige IVA al construirse.

Cálculo de ejemplo con los datos del Main

Precio base de ambos productos: $1000. IVA: 21 %. EO: ganancias impositivas de $4. EI: ganancias impositivas de $3,50. Son las fórmulas académicas del ejercicio, no un cálculo tributario para uso real.

Concepto	Operación	Resultado
IVA	1000 × 0,21	210
EO	(0,5 × 1000) / (4 × 4)	31,25
EI	1000 / 4 + 0,3 × 3,50	251,05
Electrónico	1000 + 210 + 31,25	1241,25
Hogar	1000 + 210 + 251,05	1461,05

Al cambiar IVA a 25 % y ganancias de EO a $5, el nuevo electrónico vale 1000 + 250 + 25 = $1275. La primera Venta mantiene $1241,25, porque su importe ya fue registrado.

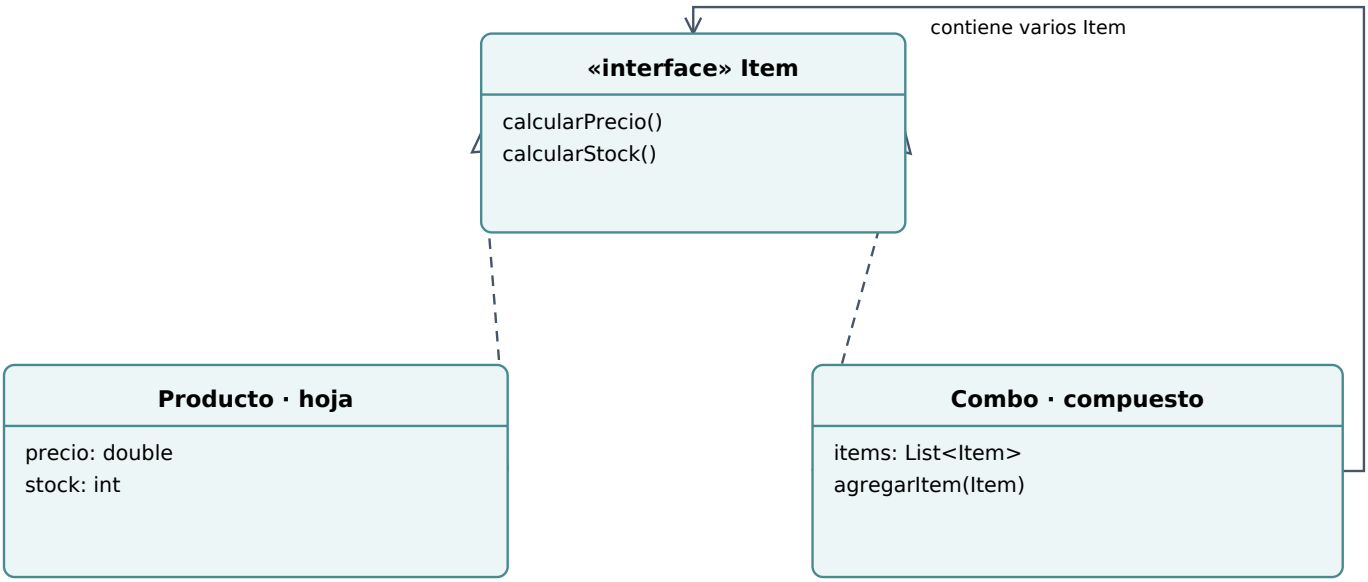
Qué justificar en la prueba

Hay Observer porque el mismo evento activa reacciones distintas mediante un contrato. Hay Adapter porque dos reacciones traducen hacia interfaces externas. La lista de Impuesto permite combinar cálculos; no es automáticamente Decorator, porque no hay capas de impuestos que envuelvan el mismo componente.

El PDF tiene inconsistencias tipográficas: menciona IVA y luego escribe INA en la fórmula; también aparece El donde corresponde EI. La solución Java usa IVA y EI. TipoProducto es una clase abstracta en el código actual, no una interfaz.

Composite modela hojas y grupos de grupos

Intención. Componer objetos como un árbol y permitir que el cliente consulte un elemento o una agrupación con el mismo contrato. Es estructural. La estructura se recorre por delegación recursiva.



Roles

Componente: Item. Hoja: Producto. Compuesto: Combo. Un Combo no contiene solamente Producto: contiene Item, por eso admite otros combos y elementos decorados.

Recursión sin preguntar el tipo

El precio de un combo se calcula sumando item.calcularPrecio() para cada hijo. Si el hijo es simple, devuelve su precio. Si es otro combo, calcula su propia suma. El padre no usa instanceof ni conoce la profundidad del árbol.

El stock del combo del ejercicio es el mínimo de sus componentes. Si casco tiene 5 unidades, guantes 54 y chaleco 3, se pueden formar 3 combos. Esa regla supone una unidad de cada componente y disponibilidades independientes.

Límites que conviene declarar

Con cantidades distintas, el stock sería el mínimo de stockDisponible / cantidadNecesaria, usando división entera. Si dos ramas consumen el mismo inventario, tomar mínimos aislados puede sobreestimar lo disponible: habría que sumar la demanda por producto. Composite organiza la estructura; no resuelve por sí solo la gestión compartida de inventario.

No permitas que un combo se contenga a sí mismo o que forme un ciclo indirecto. Eso generaría recursión infinita. Para un compuesto vacío, definí el resultado: la solución de la raíz devuelve precio 0 y stock 0.

Composite en Java y seguimiento del cálculo

Propuesta didáctica con List y ArrayList importados desde java.util. Se usa el contrato del módulo de stock del repositorio.

```
interface Item {
    double calcularPrecio();
    int calcularStock();
}
class Producto implements Item {
    private final double precio;
    private final int stock;
    Producto(double precio, int stock) {
        this.precio = precio; this.stock = stock;
    }
    public double calcularPrecio() { return precio; }
    public int calcularStock() { return stock; }
}
class Combo implements Item {
    private final List<Item> items = new ArrayList<>();
    void agregar(Item item) { items.add(item); }
    public double calcularPrecio() {
        double total = 0;
        for (Item item : items) total += item.calcularPrecio();
        return total;
    }
    public int calcularStock() {
        if (items.isEmpty()) return 0;
        int minimo = Integer.MAX_VALUE;
        for (Item item : items) {
            minimo = Math.min(minimo, item.calcularStock());
        }
        return minimo;
    }
}
```

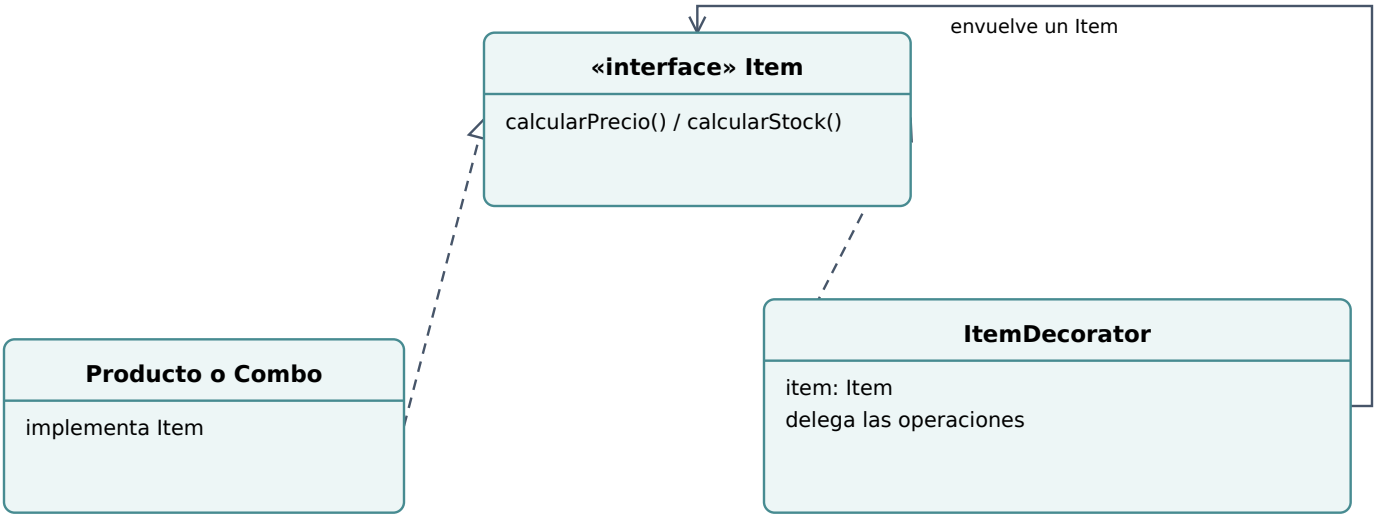
Ejercicio resuelto

A cuesta 100 y tiene stock 5. B cuesta 200 y tiene stock 3. C cuesta 50 y tiene stock 9. Combo X contiene A y B; Combo Y contiene X y C. X vale 300 y tiene stock 3. Y vale 350 y tiene stock 3. Y delega en X y C; X delega en A y B.

El cliente pide únicamente Y.calcularPrecio(). Guardar en Y un precio manual además del cálculo crearía dos fuentes de verdad. Para las tareas del ejercicio IV ocurre lo mismo con horas y estado de completitud.

Decorator agrega capas con el mismo contrato

Intención. Agregar responsabilidades a un objeto en tiempo de ejecución conservando su interfaz. Es estructural. Una capa implementa el contrato del componente y conserva una referencia a otro componente del mismo contrato.



Descuento y Packaging heredan de ItemDecorator. El descuento resta un monto; el packaging suma su costo. Ambos delegan el stock sin modificarlo. Un decorador puede envolver una hoja, un combo o incluso otro decorador.

Pensalo desde afuera hacia adentro

```
Item finalItem = new Packaging(
    new Descuento(new Descuento(combo, 100), 50), 30);
// Entrada: Packaging -> Descuento(50) -> Descuento(100)
//          -> combo
// Retorno con precio base 500: 500 -> 400 -> 350 -> 380
```

El cliente pregunta al objeto más exterior. Cada capa puede ejecutar comportamiento antes o después de delegar. La teoría también admite interceptar por completo: una capa puede bloquear una llamada según una regla. No todas las operaciones deben ser void.

El orden puede cambiar el resultado

Con precio 80, descuento 100 y packaging 50: Packaging(Descuento(producto,100),50) da 50 si el descuento se limita a cero. Descuento(Packaging(producto,50),100) da 30. Que las capas puedan ordenarse libremente no significa que sean conmutativas.

Para quitar una capa hay que actualizar la referencia exterior o reconstruir la cadena correspondiente. El patrón lo permite, pero Java no elimina automáticamente un decorador del medio de la cadena.

Decorator en Java y errores típicos

Estos fragmentos usan Item de Composite. Son una simplificación de stock_precios_java (2).

```
abstract class ItemDecorator implements Item {
    protected final Item item;
    ItemDecorator(Item item) {
        this.item = java.util.Objects.requireNonNull(item);
    }
    public double calcularPrecio() {
        return item.calcularPrecio();
    }
    public int calcularStock() {
        return item.calcularStock();
    }
}
class Descuento extends ItemDecorator {
    private final double monto;
    Descuento(Item item, double monto) {
        super(item); this.monto = Math.max(0, monto);
    }
    public double calcularPrecio() {
        return Math.max(0, item.calcularPrecio() - monto);
    }
}
class Packaging extends ItemDecorator {
    private final double monto;
    Packaging(Item item, double monto) {
        super(item); this.monto = Math.max(0, monto);
    }
    public double calcularPrecio() {
        return item.calcularPrecio() + monto;
    }
}
```

Lo que tenés que explicar del diseño

ItemDecorator es un Item y a la vez tiene un Item. La primera relación permite usarlo donde se espera el contrato; la segunda permite envolver y delegar. Si guardara solamente Producto, ya no podría decorar un Combo o un Descuento.

• Olvidar delegar pierde el comportamiento base, salvo que bloquearlo sea la intención explícita.

• Devolver un stock propio desde Packaging rompe la regla de que el envase no modifica existencias.

• Crear ProductoConDescuentoConCaja por cada combinación multiplica las subclases y pierde la composición dinámica.

Stock y precios resuelto paso a paso

Consigna. Productos simples, combos y combos de combos. Precio del combo como suma, descuentos fijos acumulables y packaging con costo. Resolución presente en dos carpetas del repositorio.

Correspondencia entre las dos versiones

Rol	stock_precios_java (2)	PruebaTincho/solución-fravega
Componente	Item, interfaz	Producto, clase abstracta
Hoja	Producto	ProductoSimple
Compuesto	Combo	Combo
Decorador base	ItemDecorator	Aplicable
Capas concretas	Descuento y Packaging	DescuentoFijo y Packaging

Aunque una carpeta se llame solución-fravega, ese código resuelve el módulo de stock y precios con Composite y Decorator. No contiene la integración contable de la otra consigna Frávega.

Resultados del Main de la raíz

Hamburguesa 5000 con stock 10; papas 2000 con stock 5; gaseosa 1500 con stock 20. El combo cuesta 8500 y su stock es 5. Al agregar Coca de 2000 con stock 10, el combo grande cuesta 10500, con stock 5.

La cadena exterior suma packaging de 300 después de descuentos de 1000 y 500: 10500 - 1000 - 500 + 300 = 9300. El stock final sigue siendo 5.

Resultados de los tests de la otra versión

Casco, chaleco y guantes valen 100 cada uno: combo 300 y stock 3. Con piloto de 100 y stock 2: combo recargado 400 y stock 2. La cadena más larga del test da 300 + 50 + 100 + 100 - 100 - 50 = 400.

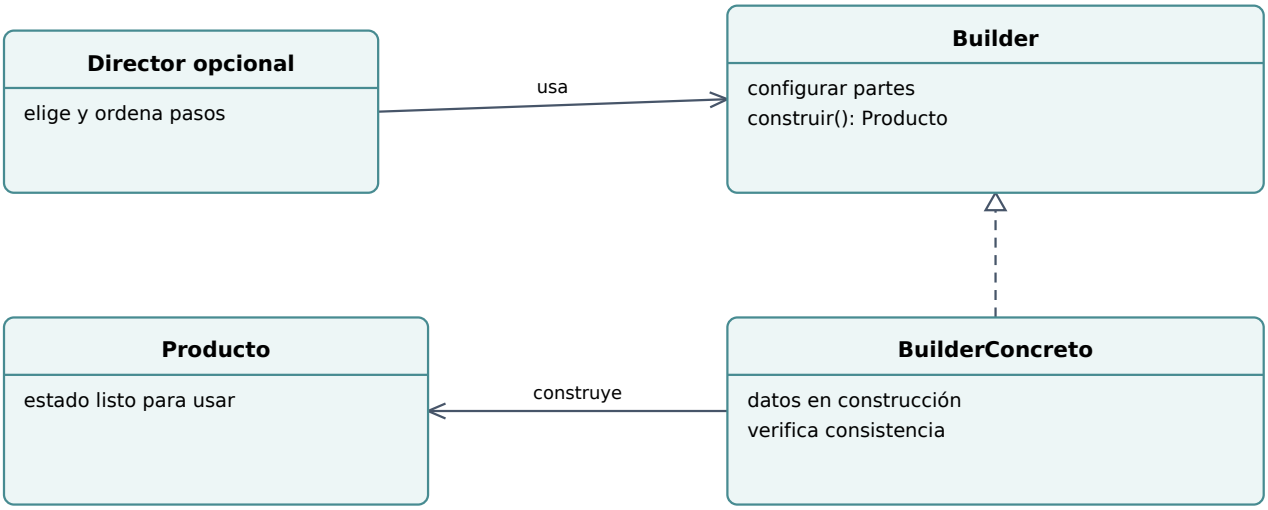
Detalles que no conviene copiar sin pensar

La versión Aplicable permite precios negativos porque DescuentoFijo resta sin limitar a cero. Su Combo.stock falla si el conjunto está vacío. La versión Item contempla ambos casos. El HashSet de la versión Aplicable evita repetir la misma referencia, pero no modela cantidades explícitas.

El gráfico textual del README ubica de forma confusa ItemDecorator bajo Combo. La relación correcta es: Producto, Combo e ItemDecorator implementan Item; Descuento y Packaging extienden ItemDecorator. Combo contiene varios Item; el decorador envuelve uno.

Builder construye un objeto válido por pasos

Intención. Separar el proceso de construcción de la representación final. Es creacional. Sirve cuando crear un objeto requiere varios pasos, muchos datos o validar combinaciones antes de entregarlo al cliente.



En la versión clásica, Builder define los pasos, los builders concretos construyen representaciones y el Director coordina una receta. En una variante fluida, un builder concreto ofrece métodos encadenables y build o construir. El Director y la interfaz no son obligatorios en todos los problemas. Para tu prueba, conviene conocer primero la estructura clásica que aparece en el apunte y después esta variante fluida.

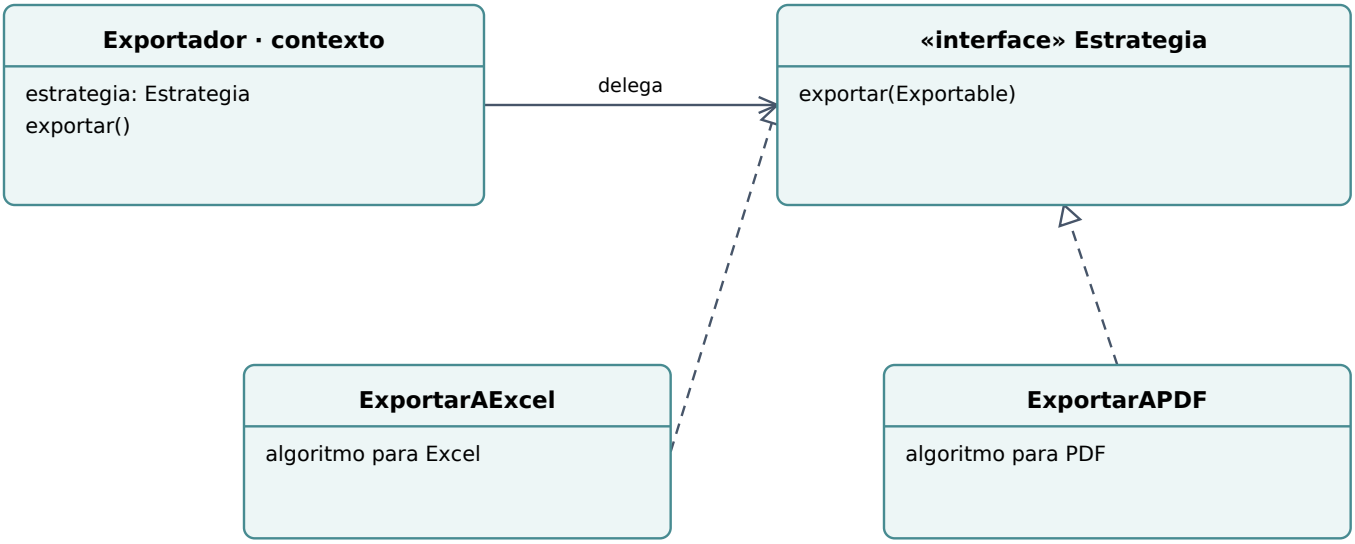
Ejemplo didáctico autocontenido

```
final class Aviso {
    private final String mensaje, destino;
    private Aviso(String mensaje, String destino) {
        this.mensaje = mensaje; this.destino = destino;
    }
    static class Builder {
        private String mensaje, destino;
        Builder mensaje(String m) { mensaje = m; return this; }
        Builder destino(String d) { destino = d; return this; }
        Aviso construir() {
            if (mensaje == null || mensaje.trim().isEmpty() ||
                destino == null || destino.trim().isEmpty()) {
                throw new IllegalStateException("Faltan datos");
            }
            return new Aviso(mensaje, destino);
        }
    }
}
// Aviso a = new Aviso.Builder().mensaje("Expensas")
//     .destino("usuario@ejemplo.com").construir();
```

La validación final impide devolver un Aviso incompleto. No basta con poner setters que devuelven this para afirmar que se resolvió la consistencia. El producto debe conservar también las invariantes que puedan romperse después de construirlo.

Strategy intercambia una forma de resolver

Intención. Encapsular algoritmos alternativos detrás de un contrato. Es de comportamiento. El contexto delega el trabajo a su estrategia actual y puede recibir otra en ejecución.




```
interface Tratamiento {
    double costo(double base);
}
class Estandar implements Tratamiento {
    public double costo(double base) { return base; }
}
class Rapido implements Tratamiento {
    public double costo(double base) { return base + 1500; }
}
class Despacho {
    private Tratamiento tratamiento = new Estandar();
    void setTratamiento(Tratamiento t) { tratamiento = t; }
    double costo(double base) { return tratamiento.costo(base); }
}
```

Un Despacho con base 2000 cuesta 2000 usando Estandar. Después de setTratamiento(new Rapido()), cuesta 3500. Cambia el algoritmo delegado; no se agrega una capa alrededor del despacho.

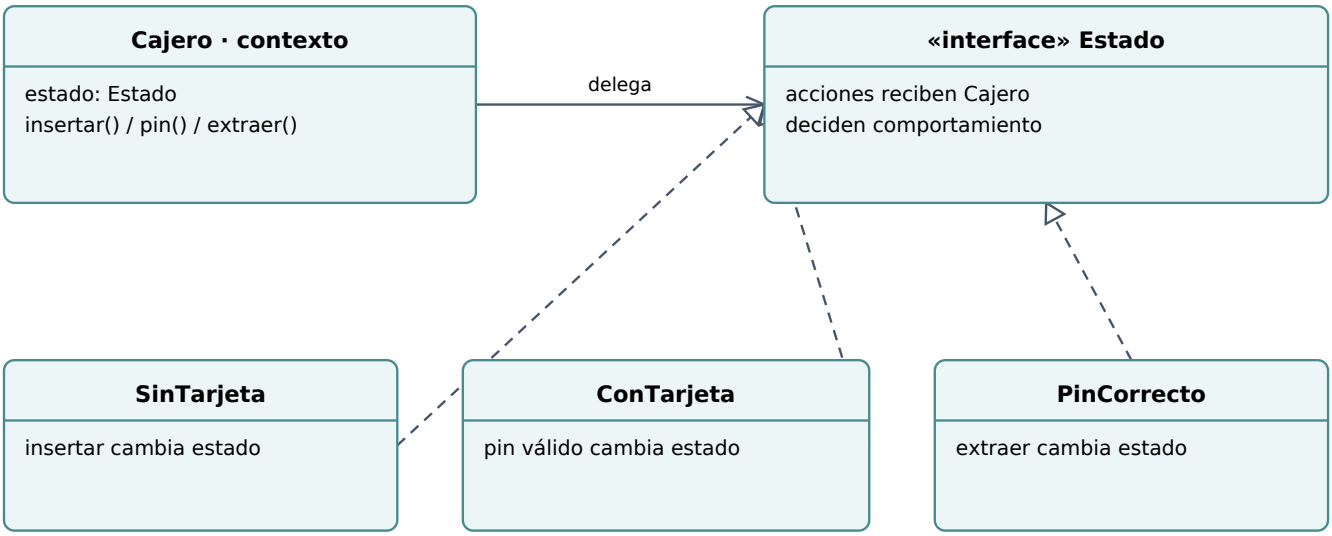
Qué recordar del apunte

Las estrategias concretas no necesitan conocerse entre sí y no definen transiciones entre ellas. Pueden tener configuración, recibir parámetros y ser reutilizables si no conservan datos incompatibles entre usos. El contrato puede ser interfaz o clase abstracta.

Señales: elegir criterio de liquidación, algoritmo de despacho o formato de exportación. Su costo es incorporar clases y configurar correctamente el contexto. No hace falta usarlo para toda variación trivial de una sola operación.

State cambia las acciones según el estado

Intención. Representar cada estado relevante con un objeto y delegar en él el comportamiento del contexto. Es de comportamiento. También define las transiciones válidas del ciclo de vida.



Transiciones del caso de cajero

Estado actual	Evento válido	Estado siguiente
SinTarjeta	Insertar tarjeta	ConTarjeta
ConTarjeta	PIN correcto	PinCorrecto
ConTarjeta	PIN incorrecto	ConTarjeta, con error
PinCorrecto	Solicitar extracción válida	Extrayendo
Extrayendo	Finalizar operación	PinCorrecto, supuesto
ConTarjeta o PinCorrecto	Expulsar tarjeta	SinTarjeta

La consigna enumera estados y acciones, pero no fija todas las transiciones. En esta propuesta, finalizar la extracción permite otra operación y expulsar termina la sesión. Durante Extrayendo se rechazan acciones incompatibles. Estas reglas son supuestos que hay que declarar.

Tres variantes presentes en la teoría

1. Estado abstracto con referencia al contexto. 2. Estado abstracto que recibe el contexto por parámetro. 3. Estado como interfaz que recibe el contexto. Las variantes sin referencia guardada pueden compartir instancias si los estados no conservan datos de una sesión particular.

Un enum con un switch puede representar estados de dominio, pero el patrón State estudiado extrae el comportamiento a clases. La ventaja aparece cuando varias acciones cambian según el estado; no se reduce a cambiar el valor de una variable.

State en Java con una transición visible

Propuesta didáctica. Se muestra el ciclo mínimo de inserción y PIN; la extracción se modela en la resolución del ejercicio. Los métodos default dan una respuesta para acciones inválidas.

```
interface Estado {
    default void insertar(Cajero c) {
        throw new IllegalStateException("Ya hay una tarjeta");
    }
    default void pin(Cajero c, boolean correcto) {
        throw new IllegalStateException("Primero inserte tarjeta");
    }
}
class SinTarjeta implements Estado {
    public void insertar(Cajero c) {
        c.cambiar(new ConTarjeta());
    }
}
class ConTarjeta implements Estado {
    public void pin(Cajero c, boolean correcto) {
        if (!correcto) throw new IllegalArgumentException("PIN");
        c.cambiar(new PinCorrecto());
    }
}
class PinCorrecto implements Estado {
    public void pin(Cajero c, boolean correcto) {
        throw new IllegalStateException("PIN ya validado");
    }
}
class Cajero {
    private Estado estado = new SinTarjeta();
    void cambiar(Estado siguiente) { estado = siguiente; }
    void insertar() { estado.insertar(this); }
    void pin(boolean correcto) { estado.pin(this, correcto); }
}
```

Recorrido

Al crear el cajero, ingresar PIN falla. insertar() delega a SinTarjeta, que cambia a ConTarjeta. Un PIN incorrecto lanza error y conserva ese estado. Un PIN correcto cambia a PinCorrecto. Una nueva llamada a pin informa que ya fue validado.

State frente a Strategy

En ambos casos hay delegación. En Strategy se elige una forma alternativa de resolver algo. En State, una acción se interpreta según la situación actual y puede disparar una transición. En este ejemplo ConTarjeta conoce el próximo estado; en Strategy las alternativas no se van pasando el control entre sí.

Módulo exportador resuelto y revisado

Consigna resumida. Exportar datos tabulares de un Map a Excel o PDF, elegidos en ejecución. Excel usa Apache POI. PDF usa inicialmente PDFBox, con posibilidad de reemplazo. Después de exportar a PDF se agrega una marca de agua.

Lo que ya implementa el repositorio

Exportador mantiene una EstrategiaDeExportacion y un Exportable. Documento implementa Exportable y devuelve Map<String,List<String>>. ExportarAExcel usa POI. ExportarAPDF delega en AdapterExportadorPDF, implementado por AdapterApachePDFBox.

```
// API real usada por los tests del repositorio:
Exportador e = new Exportador(new ExportarAExcel("datos.xlsx"));
e.setExportable(documento);
e.exportar();
e.setEstrategia(new ExportarAPDF(
    new AdapterApachePDFBox("datos.pdf")));
e.exportar();
```

Strategy: elegir Excel o PDF. Adapter: aislar PDFBox detrás del contrato propio. Cambiar de formato y cambiar de biblioteca PDF son variaciones diferentes, por eso aparecen dos niveles de colaboración.

Qué falta y cómo completarlo

La marca de agua no está implementada en los Java revisados. Propuesta: una capa ConMarcaDeAgua implementa EstrategiaDeExportacion, envuelve la estrategia PDF, primero delega la exportación y después usa un colaborador que modifica el PDF generado. Devuelve la ruta del archivo ya procesado.

```
// Pseudocódigo de la responsabilidad del decorador:
String exportar(Exportable doc) {
    String ruta = componente.exportar(doc);
    servicioMarca.agregar(ruta, "Autoría");
    return ruta;
}
```

El servicio de marca debe operar realmente sobre el PDF; imprimir un mensaje no cumple el requisito. Se configura la capa sobre PDF para evitar aplicarla a Excel. El cliente sigue trabajando con el contrato común.

Qué demuestran los tests existentes

Comparan la ruta devuelta con tmp/datos.xlsx o tmp/datos.pdf. No comprueban por sí solos que el archivo exista, sus celdas, su contenido ni una marca de agua. Además, las implementaciones capturan IOException e imprimen el error, pero aun así devuelven la ruta: no debe confundirse devolver una ruta con exportar exitosamente.

Gestión de consorcios como caso integrador

Fuente. 8-Ejercicios patrones III - Gestión de consorcios.pdf. El repositorio contiene la consigna, no una solución Java de este caso. Lo siguiente es una propuesta de diseño enfocada en los patrones de tu evaluación.

Modelo mínimo del dominio

Consorcio conoce unidades funcionales y gastos del período. UnidadFuncional tiene porcentaje, propietarios, posibles inquilinos y deuda anterior. Gasto tiene monto, tipo, categoría de expensa y recurrencia. Expensa registra fecha, importes ordinario y extraordinario y total. EjecucionLiquidacion guarda administrador, fecha y hora, período y criterio elegido.

Patrones que se pueden justificar

• Strategy para CriterioLiquidacion: el administrador elige cada mes el algoritmo. Alternativas: pago completo, uso de reservas y generación de reservas futuras.

• Adapter para consultar saldo: el dominio pide saldo de una cuenta y el adaptador traduce al componente externo que exige CBU o alias, fecha y token.

• Strategy para MedioNotificacion si se elige email, WhatsApp o SMS como alternativas de envío. Los adaptadores de proveedores pueden quedar dentro de cada estrategia.

• Observer es defendible si la generación de expensas publica un evento y hay interesados suscritos que reaccionan. Enviar un mensaje por sí solo no demuestra este patrón.

Pasos comunes del cálculo

La consigna exige obtener saldo, calcular gastos y dividir expensas. Se puede mantener los dos primeros en un servicio y delegar el reparto al criterio. Una jerarquía con un método plantilla sería otra opción: se relaciona con Template Method, fuera de los siete patrones que mencionaste. No hace falta llamar Builder a todo proceso que tenga pasos.

Ejemplo numérico con supuestos explícitos

Gastos a cubrir: 100000; saldo aplicable: 20000; sin reservas, sin deudas ni gastos particulares. Propuesta de pago completo: importe a repartir = max(0, gastos - saldo) = 80000. Unidad A: 60 %, paga 48000. Unidad B: 40 %, paga 32000. Después se notifica a A por WhatsApp y B por email, como pide el caso de la consigna.

La fórmula del ejemplo es un supuesto de resolución: el PDF no define todas las reglas contables de reservas, deudas o redondeo. No hay que inventarlas como si fueran datos confirmados. Recurrencia de gastos y confirmación manual de pagos son reglas de negocio; no requieren por sí mismas un patrón de esta lista.

El examen original también pide arquitectura y persistencia. Esos puntos amplían el alcance de aquella consigna y no prueban que formen parte de tu evaluación actual de patrones.

Patrones IV tareas y subtareas

Consigna resumida. Hay tareas simples y compuestas, con nombre, horas y completitud. Las compuestas suman horas y están completas solo si lo están todas sus subtareas. Se permite anidamiento sin un nivel fijo.

Resolución propuesta con Composite

Tarea es el componente. TareaSimple guarda nombre, horas y un booleano completa. TareaCompuesta guarda nombre y List<Tarea>; calcula horas y completitud a partir de sus hijos. El cliente usa horas() y completa() sin distinguir hojas de grupos.

```
interface Tarea {
    String nombre();
    int horas();
    boolean completa();
}
// Núcleo de TareaCompuesta:
public int horas() {
    int total = 0;
    for (Tarea t : subtareas) total += t.horas();
    return total;
}
public boolean completa() {
    for (Tarea t : subtareas) {
        if (!t.completa()) return false;
    }
    return true;
}
```

Los métodos anteriores pertenecen a una clase que implementa Tarea y tiene List<Tarea> subtareas; se muestran solo las operaciones relevantes. En esta propuesta, una tarea compuesta vacía tiene 0 horas y devuelve true, por la regla de que no existe ninguna subtarea incompleta. Si se quiere impedir ese caso, se valida que tenga hijos.

Ejercicio resuelto

Frontend contiene maqueta de 3 h completa y pantallas de 5 h incompletas. Backend es una tarea simple de 7 h completa. Deploy requiere 2 h y está completo. Sitio web contiene Frontend, Backend y Deploy.

Frontend: 3 + 5 = 8 h, incompleto. Sitio web: 8 + 7 + 2 = 17 h, incompleto. Al completar pantallas, Frontend y Sitio web pasan a completos sin modificar manualmente un booleano en sus grupos.

Qué dibujar y defender

TareaSimple y TareaCompuesta realizan Tarea. TareaCompuesta contiene varios Tarea. Evitá guardar horas y completa como valores manuales del grupo: se desactualizarían cuando cambien sus hijos. Tampoco hagas List<TareaSimple>, porque impediría las subtareas compuestas.

Patrones IV notificaciones y playlists

Notificaciones personalizadas con Decorator

Consigna. Una notificación tiene enviar(). Se agregan prioridad alta, negrita, sonido o reenvío como capas, en cualquier orden. Modelo: Notificacion es el contrato; NotificacionBase contiene el mensaje; DecoradorNotificacion implementa el contrato y contiene una Notificacion; cada personalización es un decorador concreto.

```
// Esquema del recorrido, no una implementación de transporte:
Notificacion n = new Sonido(
    new PrioridadAlta(new NotificacionBase("Aviso")));
n.enviar();
// Sonido -> PrioridadAlta -> NotificacionBase
// Las capas ejecutan acciones antes o después de delegar.
```

En una implementación real, prioridad y negrita deben llegar al mecanismo que presenta o envía el mensaje. Un println de prioridad antes de enviar no demuestra que se haya cambiado la prioridad efectiva. Reenvio puede delegar el envío original y realizar otro envío al grupo configurado.

Orden de ejecución. Si A ejecuta antes y después de delegar, y B también, A(B(base)) recorre: antes A, antes B, base, después B, después A. Este seguimiento es una pregunta frecuente de prueba.

Playlist con efectos usa dos patrones

Consigna. Una playlist puede contener canciones u otras playlists. Se aplican volumen, eco, velocidad rápida y bucle a una canción o a una playlist; los efectos se apilan.

Composite: ElementoReproducible define reproducir(); Cancion es hoja; Playlist contiene List<ElementoReproducible>. Decorator: Efecto implementa ElementoReproducible y envuelve un elemento del mismo contrato.

```
ElementoReproducible tema = new Eco(new Cancion("Tema"));
Playlist lista = new Playlist();
lista.agregar(tema);
lista.agregar(otraPlaylist);
ElementoReproducible salida = new VolumenAlto(lista);
salida.reproducir();
```

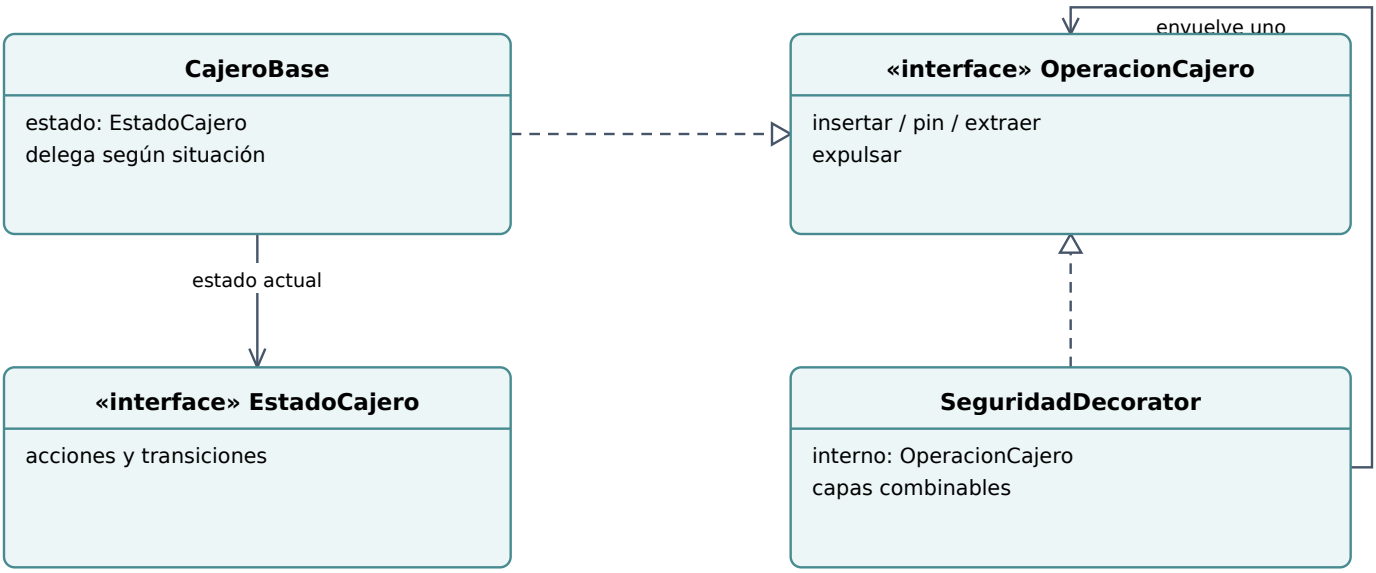
VolumenAlto envuelve la playlist entera; Eco afecta solo al tema decorado. Un Bucle debe definir cantidad o mecanismo de parada: repetir para siempre sin control puede bloquear la ejecución. Para efectos reales se delegan parámetros a un motor de audio; el patrón organiza las responsabilidades, no procesa sonido por sí solo.

Patrones IV cajero con seguridad adicional

Consigna. Estados SinTarjeta, ConTarjeta, PinCorrecto y Extrayendo. Las acciones permitidas dependen del estado. También se agregan y quitan capas de alarma, confirmación extra y silencio en ejecución.

Resolución propuesta con State y Decorator

OperacionCajero define insertar, ingresarPin, extraer y expulsar. CajeroBase implementa ese contrato y delega las acciones a EstadoCajero. SeguridadDecorator también implementa OperacionCajero y contiene otro OperacionCajero. Alarma, ModoSeguro y ModoSilencioso extienden ese decorador.



Recorrido de una operación

Con ModoSeguro(Alarma(CajeroBase)), extraer primero solicita confirmación. Si se rechaza, no delega y el cajero conserva su estado. Si se confirma, Alarma delega en el cajero; el estado verifica si la extracción es válida y, cuando corresponde, pasa a Extrayendo. La alarma actúa después según la política definida.

Cómo resolver el silencio de verdad

No alcanza con omitir un println en la capa exterior si el cajero interno imprime directamente. Propuesta: las operaciones producen una Respuesta y una interfaz de salida se encarga de presentarla. ModoSilencioso suprime esa presentación sin eliminar los controles del estado ni la operación bancaria.

Declarar si Alarma suena solo ante éxito o también ante intentos fallidos. En esta propuesta suena después de una operación exitosa. El orden de las capas determina si un intento rechazado por ModoSeguro llega a la alarma.

Patrones IV logística y tratamientos

Consigna. Envíos simples y agrupados recursivos. El grupo suma costos, genera etiquetas con detalles y usa la dirección del envío contenido de mayor peso. Tratamientos Estándar, Frágil y Rápido cambian el costo y la manipulación.

Resolución propuesta

Composite: Envio define peso(), direccion(), costo() y etiqueta(). EnvioSimple tiene peso, dirección y costo base. EnvioAgrupado contiene List<Envio> y opera sobre ellos. Strategy: TratamientoDespacho define costo(base) y manipular(); cada tratamiento implementa su algoritmo y protocolo.

```
// Núcleo de EnvioAgrupado bajo los supuestos de abajo:
double peso() {
    return hijos.stream().mapToDouble(Envio::peso).sum();
}
double costo() {
    return hijos.stream().mapToDouble(Envio::costo).sum();
}
String direccion() {
    return hijos.stream()
        .max(java.util.Comparator.comparingDouble(Envio::peso))
        .orElseThrow().direccion();
}
```

La etiqueta del grupo concatena o estructura las etiquetas de todos sus hijos. Si un hijo es otro agrupado, ese hijo resuelve a su vez su propia etiqueta y dirección.

Supuestos necesarios para no cobrar dos veces

En esta propuesta, cada envío simple calcula su costo mediante su estrategia y el agrupado suma costos finales de hijos. No se reaplica un recargo al grupo completo. Así se conserva literalmente la regla de suma del enunciado. Si se exige tratamiento global del contenedor, hay que definir otra política explícita y evitar duplicar recargos.

Para elegir dirección se comparan los hijos inmediatos por su peso total; si gana un grupo, se consulta su dirección recursiva. Si el docente pretende la hoja más pesada de todo el árbol, debe hacerse una búsqueda distinta. Se rechazan grupos vacíos y se define un criterio estable de desempate.

Ejemplo resuelto

Caja A: base 1000, peso 2, estándar: 1000. Caja B: base 2000, peso 5, frágil con seguro 100: 2000 × 1,20 + 100 = 2500. Grupo: costo 3500, peso 7, dirección de B. B indica plástico de burbujas y etiqueta Frágil. Con tratamiento rápido en B costaría 3500; el grupo costaría 4500.

Comparaciones y preguntas que suelen confundir

Comparación	Diferencia para justificar
Adapter y Decorator	Adapter traduce hacia el contrato requerido. Decorator conserva el contrato y agrega o intercepta responsabilidades.
Composite y Decorator	Composite reúne varios hijos y los trata recursivamente. Decorator envuelve un componente para agregar una capa. Ambos pueden coexistir.
Strategy y State	Strategy configura un algoritmo alternativo. State representa una situación y regula acciones y transiciones.
Strategy y Decorator	Strategy reemplaza una forma de resolver. Decorator apila responsabilidades alrededor de un objeto.
Observer y Strategy de aviso	Observer relaciona eventos e interesados. Strategy elige cómo enviar un aviso concreto.
Builder y setters	Builder coordina y valida la construcción antes de entregar el resultado. Los setters comunes no garantizan eso.

Errores frecuentes de examen

• Nombrar el patrón por una palabra aislada. Una notificación puede ser Observer, Decorator o un simple método, según el problema.

• Dibujar una interfaz sin mostrar quién la implementa y quién la guarda como atributo. El contrato no conecta los objetos por sí solo.

• Confundir herencia con contención. Combo no extiende ProductoSimple; contiene componentes.

• Escribir el cálculo en el cliente. Si el cliente recorre productos para armar el precio del combo, se pierde la responsabilidad del compuesto.

• Cambiar todos los impuestos sobre el acumulado cuando la consigna pide calcular cada uno sobre precioBase.

• Decir que un patrón garantiza rendimiento, concurrencia o confiabilidad. Su estructura no aporta automáticamente esas propiedades.

Frase útil para justificar un diseño

“El problema variable es ____. Defino el contrato ____ y delego desde ____ hacia ____. Puedo agregar ____ sin cambiar ____. El costo o límite de esta decisión es ____.” Completá con el caso real; evitá limitarte a decir que el patrón es más fácil o más ordenado.

Simulacro para resolver sin mirar

Práctica propuesta, no modelo oficial del docente. Tiempo sugerido: 60 a 75 minutos. Dibujá las clases con sus métodos y referencias antes de escribir código.

Parte A reconocimiento y justificación

1. Al confirmar una inscripción se actualiza un contador, se genera una factura y se avisa al organizador. Los interesados pueden agregarse o retirarse. Elegí un patrón y asigná sus roles.

2. El proveedor de facturación recibe emitirFactura(idFiscal, centavos), pero el dominio necesita registrar(Inscripcion). ¿Qué agregarías? Explicá la relación con el punto anterior.

3. Una carpeta contiene archivos y otras carpetas. Todos permiten calcular su tamaño. ¿Qué tipo debe tener la colección de la carpeta y por qué?

4. Un informe puede agregar numeración, firma y cifrado de forma acumulable. ¿Qué patrón sirve? ¿Da igual el orden de firma y cifrado?

Parte B seguimiento de objetos

5. A cuesta 120 y tiene stock 7; B cuesta 80 y tiene stock 4; C cuesta 50 y tiene stock 9. X contiene A y B. Y contiene X y C. Calculá precio y stock de Y. Después aplicá Packaging(Descuento(Y,300),40), con límite de precio cero.

6. Un sujeto tiene observadores A y B. Notifica una vez, elimina A, agrega C y notifica dos veces más. Indicá el total recibido por cada uno. ¿Eso demuestra ejecución paralela?

Parte C diseño integrador

7. Un pedido pasa por Borrador, Confirmado y Enviado. Solo Borrador permite agregar productos; confirmar pasa a Confirmado; despachar solo funciona desde Confirmado. Se elige envío estándar o rápido, se integran dos transportistas con APIs diferentes y se agregan seguro y seguimiento como capas opcionales. Proponé contratos y clases para cada variación.

8. Se quiere armar una Solicitud con destinatario, fecha, importe y una combinación válida de medio de contacto y datos. El objeto no debe estar disponible incompleto. Proponé un patrón y explicá dónde validarías.

Cómo corregirte

Por cada respuesta revisá intención, roles, relación UML y un recorrido de ejemplo. Si acertás el nombre pero no podés mostrar quién delega en quién, todavía falta practicar el diseño. Para el caso integrador, no hace falta forzar los siete patrones.

Resolución razonada del simulacro

1. Observer. GestorInscripciones es sujeto; ObservadorInscripcion define alConfirmar(Inscripcion); Contador, Facturacion y AvisoOrganizador son observadores. El sujeto administra las suscripciones y avisa después de confirmar.

2. Adapter. AdaptadorFacturacion implementa el contrato que espera el dominio y delega en el proveedor. Puede además ser un observador del primer punto si recibe directamente el evento. La conversión de importe a centavos se concentra en el adaptador.

3. Composite. Elemento define tamanio(); Archivo es hoja; Carpeta contiene List<Elemento>. Cada carpeta suma tamanio() de sus hijos. List<Archivo> impediría guardar carpetas dentro de carpetas.

4. Decorator. Todas las capas implementan Informe y contienen un Informe. El orden de firma y cifrado cambia qué datos se firman y qué información queda cifrada. Que se puedan combinar no significa que produzcan el mismo resultado en cualquier orden.

5. X: precio 200 y stock 4. Y: precio 250 y stock 4. Descuento(Y,300) da max(0,250 - 300) = 0; Packaging suma 40. Resultado: precio 40 y stock 4. Invirtiendo las capas: max(0,250 + 40 - 300) = 0.

6. A recibe 1; B recibe 3; C recibe 2. El patrón no implica paralelismo. En la variante enseñada, cada notificación recorre los observadores secuencialmente.

7. State para el ciclo de Pedido, con EstadoPedido, Borrador, Confirmado y Enviado. Pedido delega agregar, confirmar y despachar. Los estados rechazan operaciones inválidas y realizan las transiciones correspondientes.

Strategy para calcular o ejecutar el despacho según modalidad. Adapter para cada transportista detrás de Transportista, traduciendo sus APIs. Decorator para seguro y seguimiento sobre un contrato común de servicio de envío. La consigna no afirma que existan suscriptores: no hace falta introducir Observer.

Un recorrido posible: confirmar el pedido, configurar la estrategia rápida, envolver el servicio elegido con seguro y seguimiento, y despachar desde Confirmado. El estado decide si se puede; la estrategia decide cómo se despacha; el adaptador traduce la integración; las capas agregan responsabilidades.

8. Builder. SolicitudBuilder recibe datos por pasos y construir verifica campos obligatorios, importe válido y compatibilidad del medio de contacto con sus datos. Solo devuelve Solicitud si está consistente. Un constructor privado y atributos finales ayudan a conservar esas garantías. Si el dominio permite cambios posteriores, también hay que validarlos.

Autoevaluación

Si fallaste en 1 o 6, repasá Observer. En 2, Adapter. En 3 o 5, Composite y Decorator. En 7, compará la intención de cada patrón. En 8, diferenciá construcción válida de edición posterior del objeto.

Repaso oral y fuentes del material

Siete respuestas que deberías poder dar

• Observer: el sujeto conoce el contrato de sus observadores y los avisa ante un evento. El evento puede llevar los datos necesarios para reaccionar.

• Adapter: traduce la interfaz de una dependencia al contrato que espera mi cliente. El código específico de la dependencia queda encapsulado allí.

• Decorator: mantiene el contrato y envuelve un componente; permite apilar responsabilidades. La llamada entra por la capa exterior.

• Composite: comparte un contrato entre hojas y compuestos; el compuesto conserva componentes y resuelve de forma recursiva.

• Builder: separa y valida el proceso de construcción antes de entregar un objeto listo para usar.

• Strategy: delega un algoritmo intercambiable; las alternativas no necesitan transiciones entre sí.

• State: delega el comportamiento según la situación actual y representa las transiciones permitidas.

Última comprobación antes de entregar un ejercicio

¿Mi colección admite grupos anidados? ¿El decorador envuelve el contrato y no una sola clase concreta? ¿La sucursal depende de APIs externas o de observadores? ¿Mis algoritmos son alternativas o estados? ¿Indiqué qué pasa ante entradas inválidas, grupos vacíos y operaciones no permitidas? ¿El diagrama coincide con los atributos y con las llamadas de mi explicación?

Fuentes y alcance de revisión

Repositorio: https://github.com/GaryPoui/Ejercicios-Java. Revisión fijada al commit 4aa08b79e08552d71d727bbc8e99c7cd12f66f54.

Teoría: Patrón Observer.pdf, Patrón Adapter.pdf, Patrón Decorator.pdf, Patrón Composite.pdf, Patrón Builder.pdf, Patrón Strategy.pdf y Patrón State.pdf. Los siete apuntes indican autoría de Ezequiel Escobar. Se revisaron texto y gráficos.

Consignas: 5-Módulo de manejo de stock y precios (3).pdf; 6-Frávega (1).pdf; 7-Modulo exportador.pdf; 8-Ejercicios patrones III - Gestión de consorcios.pdf; 9-Ejercicios patrones IV.pdf.

Código y pruebas leídos: todos los Java de stock_precios_java (2), Fravega_Java (2)/fravega, PruebaTincho/solución-fravega/ddsCompoYDec/src y PruebaTincho/solucion-modulo-exportador/src/main y src/test. También README.md y configuración del proyecto exportador, que declara Java 8, POI 4.1.2 y PDFBox 2.0.19. Las versiones explican el código del ejercicio y no son una recomendación de dependencias actuales. Se inspeccionaron los duplicados de teoría y consignas por identidad de contenido.

Los ejemplos nuevos, las ampliaciones y las resoluciones sin Java en el repositorio son propuestas didácticas. Los resultados numéricos se verificaron por cálculo; no se ejecutaron las suites Java. Esta guía organiza el alcance que informaste y no anticipa preguntas reales del docente.